腾讯云安全 | [TCH] 腾讯云黑客松 Tencent Cloud Hackathon | 云鼎实验室 | 腾讯安全众测

锋刃无影 智御未来

产品视角下的安全革新：AI漏洞猎人如何破解代码漏洞挖掘痛点

丁皓

产品总监

北京云起无垠科技有限公司

2025.11

锋刃无影
智御未来

目录

CONTENTS

PART 01 背景洞察：
从漏洞挖掘实践到行业痛点分析

PART 02 AI 漏洞猎人：
产品化实践与挑战

PART 03 未来已来：
演进路线与合作机会

PART 01：背景洞察

从漏洞挖掘实践到行业痛点分析

关于我们：专注漏洞挖掘，成果显著

锋刃无影
智御未来

3年

公司成立

2个

漏洞挖掘类产品

800+

0 day 漏洞累计

行业现状：为什么需要 AI 漏洞猎人？

锋刃无影
智御未来

工具瓶颈

SAST

Fuzz

>30%
误报率

<50%
高价值漏洞检出率

动态运行 测试效率
工具自动化程度低

效率瓶颈

>400个
平均报告风险数

>30min
漏洞平均验证时长

>60 min
漏洞平均修复时长

新挑战： AI Coding 新范式下的安全压力

x10
编码速度

逻辑类↑
漏洞更复杂

PART 02：AI 漏洞猎人

产品化实践与挑战

AI 漏洞猎人：代码漏洞挖掘智能体

锋刃无影
智御未来

产品定位

模拟资深安全专家的漏洞挖掘思维

解决代码高危漏洞挖掘经验化、处置效率低的问题

产品价值观

可追溯

可复现

可修复

落地挑战： demo流程跑通 vs. 产品效果稳定

锋刃无影
智御未来

挑战一：如何确保结果的稳定性?

- 同一个项目每次跑出的结果不一致
- 同一个漏洞每次提炼的关键信息不一致，如类型、等级

挑战二：如何平衡结果有效性与分析效率?

- 上一版结果很准，但是分析时长太长了
- 这一版速度很快，本该发现的问题没有分析出来

挑战三：如何平衡模型成本与结果有效性?

- 分析一个 50w 行代码的项目需要至少上千万的 Token，有点贵

AI 漏洞猎人：架构与数据流程

锋刃无影
智御未来

源码接入
- Git 仓库
- 本地包

代码预处理
- AST 分析
- 依赖关系梳理

漏洞分析
- 风险分析
- 漏洞研判

核心处理层 (CortexAgent)

任务规划智能体

工程理解智能体

风险分析智能体

智能体协同中心
(A2A)

威胁建模智能体

报告总结智能体

文件系统 (mcp_tools)

- 代码检索引擎
- 项目记忆模块

能力分发
- SaaS云服务
(无极 AI 平台)
- 私有化部署
- 命令行工具
(vulnex)

资源管理
- 数据隔离
- 权限控制
- 任务队列

基础设施：代码预处理与搜索机制

锋刃无影
智御未来

预处理

业务领域Graph

仓库元信息

Readme、config、文档

功能点

功能-入口API

资产

密钥、文件、DB

Code Graph（异步构建）

AST

CFG

DFG

搜索机制

关键词/正则检索

ripgrep

LSP

astgrep

功能语义检索

功能模块Embedding

功能-代码关系型索引库

基础设施：项目记忆与共享机制

锋刃无影 智御未来

项目记忆

- 📁 代码架构层
  - 模块结构
  - 依赖关系
  - 技术栈分析
- 🔍 业务分析层
  - 威胁场景映射
- 📊 任务执行层
  - 执行状态总览
  - 任务依赖图
  - 性能指标汇总
- 🔗 关联关系层
  - 代码-威胁关联
  - 任务-问题关联

威胁场景映射

- 风险评估
  - 漏洞类型
  - 影响范围 (affected files/lines)
  - 攻击向量
- 代码上下文
  - 代码位置映射
  - 威胁传播路径
  - 历史记录

任务执行记忆 (可观测性)

- 任务节点
  - 任务ID & 元数据
  - 执行状态 (pending/running/completed/failed)
  - 依赖关系 (parent/child tasks)
  - 执行结果 & 日志(steps/rep-step/tokens)
- 任务流
  - 执行路径
  - 状态转换
  - 性能指标 (steps/rep-step/tokens)

核心能力：四步漏洞挖掘流程

锋刃无影 智御未来

【1】工程理解智能体（建立项目全局认知）

业务领域分析

- 技术栈
- 目录结构
- 关键配置
- 业务领域

业务分析产物

- 业务领域
- 入口
- 核心代码
- 涉及资产

程序分析

- AST
- 变量提取
- 调用链路构建

程序分析产物

- 函数列表
- 变量依赖
- 调用链路

【2】威胁建模智能体（哪里可能有问题？）

- 通用攻击场景匹配
- 逻辑漏洞场景假设
- 潜在资产危害评级
- 威胁建模任务清单

【3】风险分析智能体（真的能被利用吗？）

- 可达性分析
- Sink分析
- 防护检查点分析
- PoC 构建

【4】报告总结智能体（结构化产物交付）

- 漏洞详细报告
- 项目整体报告

落地挑战： demo流程跑通 vs. 产品效果稳定

锋刃无影
智御未来

挑战一：如何确保结果的稳定性?

- “T 型”规划策略： 威胁建模 + 风险研判
- 做“选择题”，少做“主观题”

挑战二：如何平衡结果有效性与分析效率?

- 分析策略：深度模式和快速模式、规则匹配+LLM推理
- 检索策略：少做语义，回归代码检索的本质

挑战三：如何平衡模型成本与结果有效性?

- 项目记忆共享：“分层存储+掩码机制”，如采用‘业务模块Directory-> File-> Line’
- 大小模型协同： 小模型做信息总结，大模型做推理验证

猎杀时刻：AI 漏洞猎人真实项目案例

锋刃无影
智御未来

[redacted]

漏洞案例分析： nopCommerce 优惠券业务折扣使用限制竞态条件绕过漏洞

锋刃无影 智御未来

【1】工程理解智能体

```
**项目画像**:
*   这是一个基于 **C# / ASP.NET Core** 技术栈的电商平台。
*   核心业务领域为 **【交易与促销域】(Order & Promotion Domain)**。
**关键代码映射**:
*   **折扣规则管理**: `DiscountService.cs` (核心服务，负责定义和检查折扣规则)。
*   **订单处理流程**: `OrderProcessingService.cs` (核心业务流，负责下单、扣款、应用折扣)。
*   **数据持久化**: 依赖 `DiscountUsageHistory` 表来记录优惠券的使用情况。
**业务领域分析产物**:
*   **业务领域 A:【交易与促销域】(Critical)**
    *   **特征**: 涉及资金流转、状态变更，对并发一致性要求极高。
    *   **入口 (Entry Points)**:
        *   API: `/api/order/place` (下单)
        *   Controller: `CheckoutController.ConfirmOrder()`
        *   Service: `OrderProcessingService.PlaceOrderAsync()`
    *   **核心代码 (Critical Code)**:
        *   规则检查: `DiscountService.ValidateDiscountAsync()`
        *   历史记录: `DiscountService.GetAllDiscountUsageHistoryAsync()`
        *   状态更新: `OrderProcessingService.InsertDiscountUsageHistoryAsync()`
    *   **涉及资产 (Assets)**:
        *   `Discount.LimitationTimes` (使用次数上限 - 核心资产)
        *   `DiscountUsageHistory` (使用记录表 - 关键状态)
        *   `Customer.Id` (用户身份)
        *   `Order.Total` (订单金额 - 直接受影响资产)

*   **业务领域 B:【身份与鉴权域】(High)**
    *   **特征**: 管理用户凭证、会话状态，是系统的安全边界。
    *   **入口 (Entry Points)**:
        *   Controller: `CustomerController.Login()`, `CustomerController.Register()`
        *   Service: `CustomerRegistrationService.ValidateCustomer()`
    *   **核心代码 (Critical Code)**:
        *   认证: `AuthenticationService.SignInAsync()`
        *   权限: `PermissionService.AuthorizeAsync()`
    *   **涉及资产 (Assets)**:
        *   `CustomerPassword` (用户哈希密码)
        *   `CustomerRole` (用户权限角色)

*   **业务领域 C:【内容与配置域】(Medium)**
    *   **特征**: 处理动态配置加载、插件管理及静态资源。
    *   **入口 (Entry Points)**:
        *   Controller: `PluginController.UploadPlugin()`
        *   Service: `SettingService.SaveSetting()`
    *   **核心代码 (Critical Code)**:
        *   文件操作: `PluginManager.ExtractPlugin()`
        *   配置读写: `SettingService.SetSettingAsync()`
    *   **涉及资产 (Assets)**:
        *   `appsettings.json` (全局配置)
        *   `Plugins/` (可执行代码目录)
**业务逻辑流**:
*   下单 (`PlaceOrder`) -> 检查折扣 (`ValidateDiscount`) -> 扣款 -> 记录折扣历史 (`InsertDiscountUsageHistory`)。
```

【2】威胁建模智能体

```
**攻击面识别**:
*   **入口**: `OrderProcessingService.PlaceOrderAsync` (下单接口)。
*   **敏感资产**: `Discount.LimitationTimes` (优惠券使用次数限制)。
*   **高危操作**: `InsertDiscountUsageHistoryAsync` (写入使用记录)。
**漏洞假设 (Hypothesis)**:
*   **假设 A (当前聚焦)**: 系统在“检查优惠券是否超限”和“记录优惠券已使用”之间可能存在时间差 (竞态条件)。
    *   **场景推演**: 如果在极短时间内 (并发) 发起多次下单请求，第一次请求的“使用记录”还没写入数据库，第二次请求的“检查”就已经开始了。此时 `usedTimes` 尚未增加，导致第二次请求也能通过检查。
*   **假设 B (通用风险)**: 订单金额计算可能信任了前端传入的参数，或存在负数/精度漏洞。
    *   **场景推演**: 攻击者通过代理工具修改请求中的 `item_price` 或 `discount_amount` 为负数，后端若无校验直接计算总价；或在计算过程中因浮点数精度丢失导致金额偏差。
*   **假设 C (通用风险)**: 库存扣减逻辑同样遵循 Check-Then-Act 模式，存在超卖风险。
    *   **场景推演**: 高并发抢购场景下，多个请求同时通过库存检查 (`qty > 0`)，导致最终库存变为负数。
*   **假设 D (通用风险)**: 订单查询接口可能存在越权访问 (IDOR)。
    *   **场景推演**: 遍历 `orderId` 参数，如果后端仅检查订单是否存在而未校验归属用户，可导致隐私泄露。

**输出任务清单 (Audit Matrix)**:
*   **Task 01 (P0)**: 审计 `DiscountService.cs` 第 556 行附近的 `GetAllDiscountUsageHistoryAsync` 调用。
    *   **Focus**: 检查优惠券检查与写入操作的原子性与互斥锁。
*   **Task 02 (P0)**: 审计 `OrderProcessingService.cs` 第 1440 行附近的 `InsertDiscountUsageHistoryAsync` 调用。
    *   **Focus**: 确认该操作是否被包含在主订单事务中。
*   **Task 03 (P1)**: 审计 `OrderTotalCalculationService.cs` 及相关 ShoppingCart 逻辑。
    *   **Focus**: 检查金额计算的数据源是否强制来自数据库 (而非前端参数)；检查是否存在负数金额校验。
*   **Task 04 (P1)**: 审计 `ProductService.AdjustInventory`。
    *   **Focus**: 检查库存扣减 SQL 是否使用了原子更新语句 (`UPDATE ... WHERE qty > 0`)。
*   **Task 05 (P2)**: 审计 `OrderController.Details(int orderId)`。
    *   **Focus**: 检查代码中是否包含 `order.CustomerId == currentCustomer.Id` 的鉴权逻辑。
```

【3】分析分析智能体

```
**深度聚焦与验证**:
*   **代码级证据**:
    *   **Source**: `DiscountService.cs` 行 556 读取了 `usedTimes`。
    *   **Sink**: `OrderProcessingService.cs` 行 1440 写入了 `DiscountUsageHistory`。
    *   **Gap**: 两者不在同一个函数内，且没有看到 `lock` 关键字或数据库事务 (Transaction) 包裹整个流程。
*   **约束求解**:
    *   是否存在数据库唯一索引约束? (通常 `DiscountUsageHistory` 允许同一用户多次使用，除非是“每人限一次”，否则数据库层面无法拦截)。
    *   是否存在 Redis 分布式锁? (原生 nopCommerce 代码中此处通常无锁)。
**利用路径构造 (POC 思路)**:
*   **工具**: Python `asyncio` + `aiohttp`。
*   **步骤**:
    1.  注册一个新用户。
    2.  找到一个限制使用 1 次的优惠券代码。
    3.  构造下单请求 Payload。
    4.  使用 `asyncio.gather` 在 100ms 内并发发送 10 个下单请求。
*   **预期结果**: 10 个请求全部下单成功，后台 `DiscountUsageHistory` 增加了 10 条记录，但限制明明是 1。
**结论**: **True Positive** (确认存在风险)。
```

落地成效：评估工程与挖掘成果

锋刃无影
智御未来

结果有效性

53个漏洞

0 Day 漏洞成果
（26个目标）

>90%

已知漏洞召回率
（50个靶场）

结果稳定性

> 85%

三次结果一致性

时间成本

<15min

10w行以内

<30min

100w行以内

模型成本

500万 + 200万

商业大模型 Token
（50w左右代码量）

本地小模型 Token
（50w左右代码量）

数据条件：并行10个分析任务

PART 03：未来已来

演进路线与合作机会

演进路线与合作机会

锋刃无影
智御未来

演进路线
（能力闭环）

社区共创

- MVP阶段 2025.11
- 社区开源 2025.12
- 漏洞验证 2026.02
- 漏洞修复 2026.05

THANKS

无极 AI 平台
https://ailab.clouditera.com

Hugging Face: SecGPT
https://huggingface.co/clouditera/secgpt